

Laravel Installation Guide

PHP 8.4+, Composer 2.8+, MySQL, and Laravel 13+

Learning Goals

- Check that PHP, Composer, MySQL, and Laravel meet the course version requirements.
- Apply the course database settings (`DB_DATABASE` , `DB_USERNAME` , `DB_PASSWORD`).
- Create the `student-api` project and start the Laravel server.

Supported Environments

This setup supports:

- macOS with Homebrew;
- Ubuntu 22.04 or 24.04, including WSL2;
- PHP 8.4.0 or newer;
- Composer 2.8.0 or newer (major version 2);
- Laravel 13.0.0 or newer.

Be sure to use the correct version.

Run all commands in a terminal.

- On WSL2, run them **inside Ubuntu**, not in PowerShell or Command Prompt unless a slide explicitly says otherwise.

Environment Variables

Laravel and the course setup scripts use the same variable names:

- `DB_DATABASE`
- `DB_USERNAME`
- `DB_PASSWORD`

If your code uses other names (for example, `DB_NAME` or `DB_USER` from earlier versions of this course), please rename them to match.

Open the Setup Directory

From the `ase230` course directory:

```
cd "module2/code/1_Laravel/2. Laravel Installation and Project Structure"
```

The scripts use paths relative to this directory. Return here before beginning each numbered step.

Step 0 - Version Check

In the `0_version_check` directory:

```
bash version_check.sh
```

```
=== Version Check (mac) ===
```

```
ERROR: PHP 8.4.0 or newer is required; found PHP 8.3.35.
```

```
PHP executable: /opt/homebrew/bin/php
```

```
Hint: install PHP 8.4 with Homebrew: brew install php@8.4
```

```
Hint: run 1_pre_install/ensure_install_packages.sh before continuing.
```

```
i PHP extension check skipped because PHP is unavailable or too old.
```

```
✅ Composer version OK: 2.10.3
```

```
i Laravel version check skipped; no project directory was provided.
```

```
❌ 1 check(s) failed.
```

If your PHP is not 8.4.0 or above, you need to reinstall the PHP.

Installation Flow

Step 1: Runtime and database

PHP 8.4+ → extensions → MySQL → studentdb

Step 2: Laravel tooling

Composer → shared version checks → WSL2 networking

Step 3: Application

student-api → Laravel check → .env → migrations

Run the steps in order on a new machine.

Course Database Settings

The scripts use one shared database configuration:

```
Database: studentdb
User:     ase230
Password: ase230pass
Host:     127.0.0.1
Port:     3306
```

`ensure_install_packages.sh` creates the database and user.

- `run1-2.sh` writes the same values to `student-api/.env`.

Changing the Database Values

If your course environment needs different values, change **both** files before running the setup:

- `1_pre_install/ensure_install_packages.sh`
 - `DB_DATABASE` , `DB_USERNAME` , `DB_PASSWORD`
- `run1-2.sh`
 - `DB_DATABASE` , `DB_USERNAME` , `DB_PASSWORD`

The values must match. Otherwise Laravel migrations will fail even though PHP and Composer are installed correctly.

Destructive Utilities

Do not use these for normal installation:

- `ensure_delete_packages.sh` removes PHP, MySQL, configuration, and **all local MySQL databases** after explicit confirmation.
- `reset_db.sh --mysql` drops and recreates `studentdb`, then imports `schema.sql`.

When you need to reinstall PHP/MySQL for any issues, back up important data first.

- `backup_db.sh` to backup DB with password.
 - `bash backup_db.sh studentdb`
 - It will generate sql files for backup.
- `restore_db.sh` to restore the backup db.
 - `bash restore_db.sh studentdb1232_2323.db`

Step 1: Install Runtime and MySQL

From the setup directory:

```
cd 1_pre_install  
bash ensure_install_packages.sh  
cd ..
```

This is optional installation.

- Skip this when you have PHP8.4 or later.
This is the authoritative runtime setup.
- Do not skip it on a new environment.

What Step 1 Does

`ensure_install_packages.sh` :

- detects macOS or Ubuntu;
- installs PHP 8.4+ and required extensions;
- installs and starts MySQL;
- creates `studentdb` and the `ase230` account;
- verifies PHP, extensions, MySQL, and the database with a real query.

Rerunning it is safe: `brew install` and `apt-get install` are already idempotent, so running the script again does not break a working setup.

Step 1 on macOS

The script installs Homebrew if necessary. It then always:

- installs and links `php@8.4` ;
- installs MySQL;
- starts MySQL with `brew services` .

```
brew install php@8.4  
brew link --overwrite --force php@8.4
```

Homebrew may request normal macOS administrator confirmation during setup.

Step 1 on Ubuntu / WSL2

The script always:

- adds `ppa:ondrej/php` ;
- installs versioned PHP 8.4 packages and extensions;
- selects `/usr/bin/php8.4` with `update-alternatives` ;
- installs and starts MySQL.

It uses `sudo` , so Ubuntu may ask for your Linux account password.

Step 2: macOS

From the setup directory:

```
cd 2_laravel_install  
bash mac_install.sh  
cd ..
```

The script requires Step 1's PHP 8.4+ (it stops with a message if the version is not correct), installs Composer via Homebrew, and runs the shared checker

- without a project argument. It does not create a Laravel project or modify MySQL.

Step 2: Ubuntu / WSL2

From the setup directory:

```
cd 2_laravel_install  
bash wsl2_install.sh  
cd ..
```

Requires PHP 8.4+ from Step 1.

- Installs Composer 2, runs pre-install checks, and configures WSL2 networking.

Restart WSL2 When Requested

If `make_wslconfig.sh` creates or changes `.wslconfig`, open a **Windows** PowerShell or Command Prompt and run:

```
wsl --shutdown
```

Then reopen Ubuntu and return to the setup directory.

- This restart is only needed when the WSL configuration changes.

What the Shared Check Verifies

`0_version_check/version_check.sh` reports all detected problems:

- PHP 8.4.0 or newer;
- required extensions, including `pdo_mysql`, `zip`, and `intl`;
- Composer 2.8.0 or newer;
- Laravel 13.0.0 or newer when a project directory is provided.

- Step 2 calls the checker without a project argument, so it checks only the runtime and Composer.
- Step 3 supplies `student-api` explicitly after creating the project and therefore enforces Laravel 13+ at the correct time.

Step 3: Create the Laravel Project

From the setup directory:

```
bash run1-2.sh
```

1. backs up an existing `student-api/` directory with a timestamp;
2. creates the current Laravel application skeleton;
3. requires Laravel 13+ with the shared checker;
4. writes the MySQL values to `.env` ;
5. runs `php artisan migrate --force` .

composer create-project

Use Composer's `create-project` command to create a Laravel application in the target directory:

```
TARGET_DIR="student-api"
```

```
composer create-project laravel/laravel "$TARGET_DIR"
```


Existing Projects Are Preserved

If `student-api/` already exists, it is moved—not deleted:

```
student-api_backup_YYYYMMDD_HHMMSS/
```

The script then creates a new `student-api/`. Review or remove old backup directories yourself after confirming that the new project works.

Verify the Completed Setup

```
php --version  
composer --version  
mysql --version  
bash 0_version_check/version_check.sh student-api  
  
cd student-api  
composer check-platform-reqs  
php artisan migrate:status
```

Expected minimums: PHP 8.4.0 and Laravel 13.0.0.

Start Laravel

From `student-api/` :

```
# macOS  
php artisan serve --port=8080  
  
# WSL2  
php artisan serve --host=0.0.0.0 --port=8080
```

Open <http://localhost:8080>. Stop the development server with **Ctrl+C**.

- MySQL is a separate service and must remain running.

Troubleshooting: Version Check Fails

Run the checker from the setup directory:

```
bash 0_version_check/version_check.sh student-api
```

- PHP below 8.4 or missing extension: rerun Step 1.
- Composer missing or below 2.8: rerun the correct Step 2 script.
- Laravel below 13: recreate the project with `run1-2.sh` after preserving any work you need.

Troubleshooting: MySQL

After restarting WSL2, MySQL may need to be started:

```
sudo service mysql start
```

Test the same account Laravel uses:

```
mysql -h 127.0.0.1 -u ase230 -p studentdb -e 'SELECT 1;'
```

Enter `ase230pass` unless you changed the course database values.

Troubleshooting: Migrations

If Laravel reports a missing table, run from `student-api/` :

```
php artisan config:clear  
php artisan migrate
```

`Could not open input file: artisan` means you are not inside the Laravel project directory. Do not rerun `run1-2.sh` merely to create a missing table; that command creates a fresh project and moves the existing one aside.